Klaxon v2

Une application de covoiturage entre villes

Raphaël Picoche · Devoir bilan · Développeur Web et Web Mobile

Ce que je vais vous présenter

1. Le projet et son contexte
2. Le front-end : maquettes, interfaces, partie dynamique
3. Le back-end : deux bases de données et règles métier
4. La sécurité, le jeu d’essai et la veille
5. Démonstration, limites et bilan

CONTEXTE

Refaire une application existante

Avant

Touche pas au Klaxon

Covoiturage entre agences d'une entreprise, en PHP, MySQL et Bootstrap.

Après

Klaxon v2

React, Node et Express, MySQL pour les données liées, MongoDB pour les avis.

Les nouveautés : **villes avec suggestions**, **réservation** et **avis notés**.

BESOINS

Trois actions pour l’utilisateur

1

Chercher ou proposer un trajet

Filtres par ville et par date, suggestions de villes.

2

Réserver une place

Pas sur son propre trajet, ni s’il est complet ou déjà parti.

3

Noter le conducteur

Une note de 1 à 5, réservée aux passagers du trajet.

Un seul type de compte : le même utilisateur peut être conducteur ou passager.

COMPÉTENCES

Les six compétences du devoir

Front	Compétence	Dossier
1	Maquetter des interfaces	Section 5
2	Réaliser des interfaces statiques	Section 6
3	Développer la partie dynamique	Section 7
Back		
4	Mettre en place une base relationnelle	Section 8
5	Accéder aux données SQL et NoSQL	Sections 9 et 10
6	Développer les composants métier	Section 11

ARCHITECTURE

Le navigateur passe toujours par l'API

Navigateur

Interface React

API

Node et Express : règles métier et sécurité

MySQL

Comptes, villes, trajets, réservations

MongoDB

Les avis et les notes

ENVIRONNEMENT DE TRAVAIL

Les outils utilisés

Figma
Maquettes web et mobile

VS Code
Écrire le code

Git et GitHub
Une branche par fonctionnalité

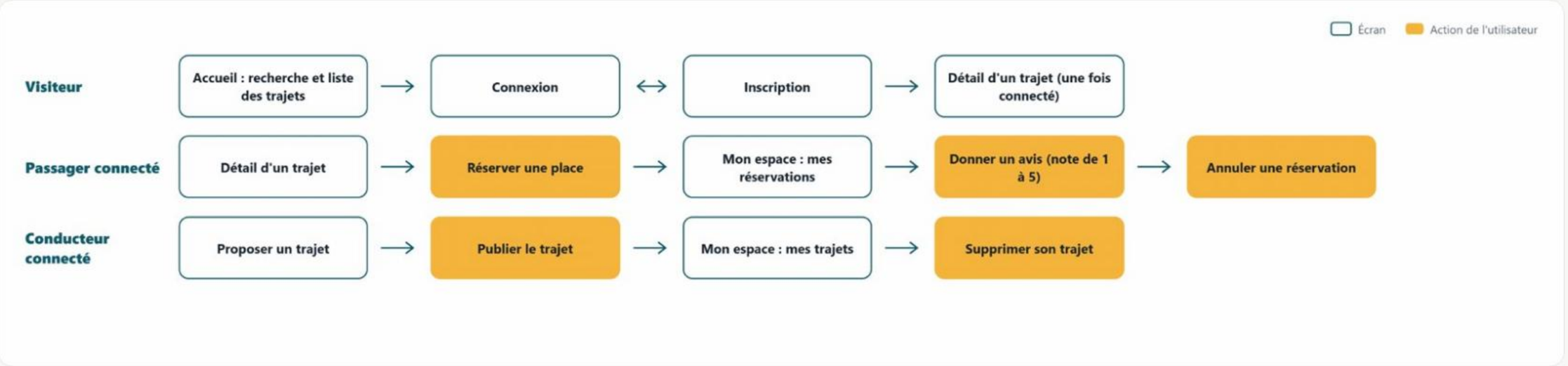
Postman
Tester l'API

phpMyAdmin et Compass
Consulter MySQL et MongoDB

Chrome
Mode mobile et cookies

FRONT-END · COMPÉTENCE 1

Maquetter avant de coder



6 écrans web

5 écrans mobile

3 parcours

FRONT-END · COMPÉTENCE 2

Des pages faites de composants

Pages

Accueil, Détail, Connexion, Inscription, Proposer, Mon espace

Composants

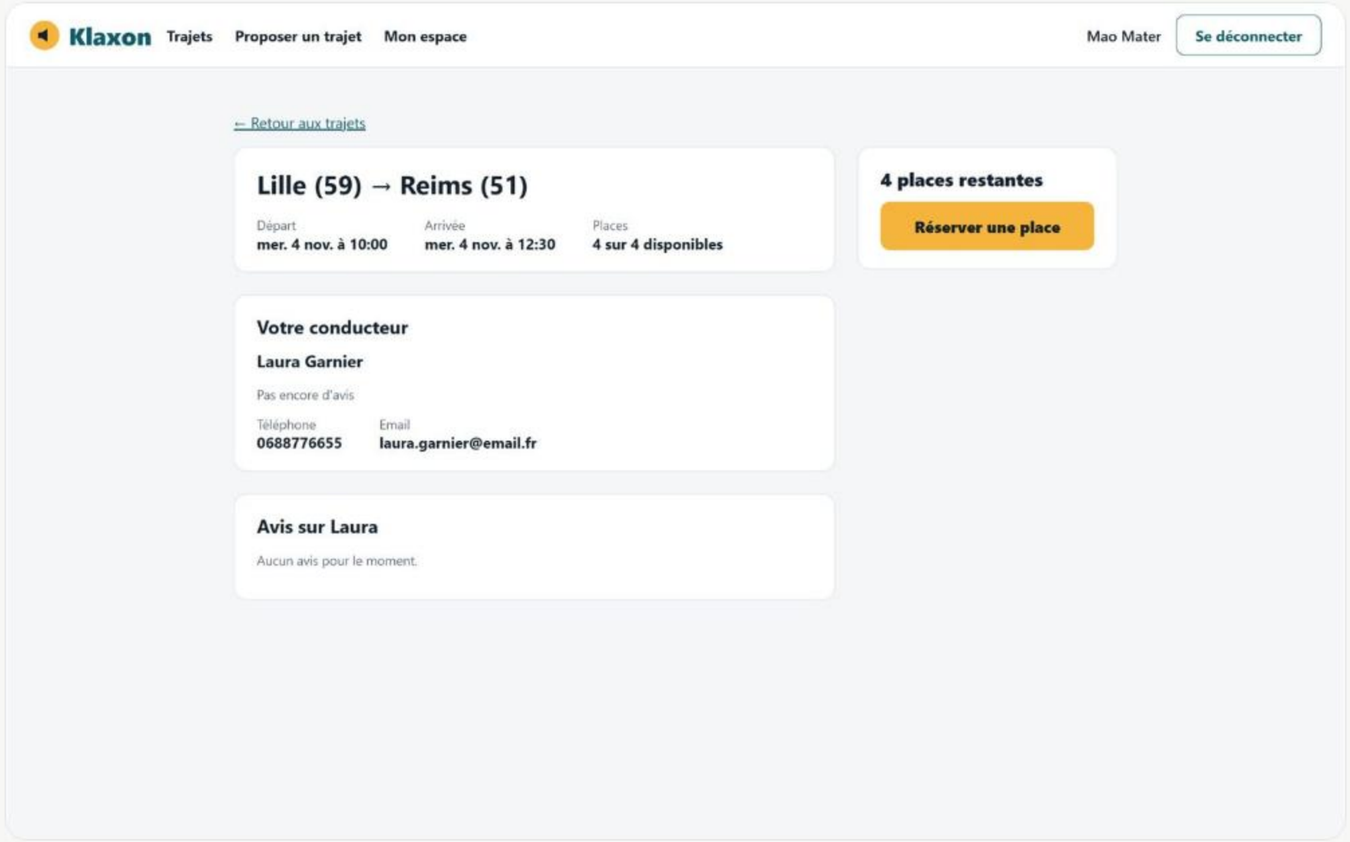
Menu, ligne de trajet, champ de ville, formulaire d’avis

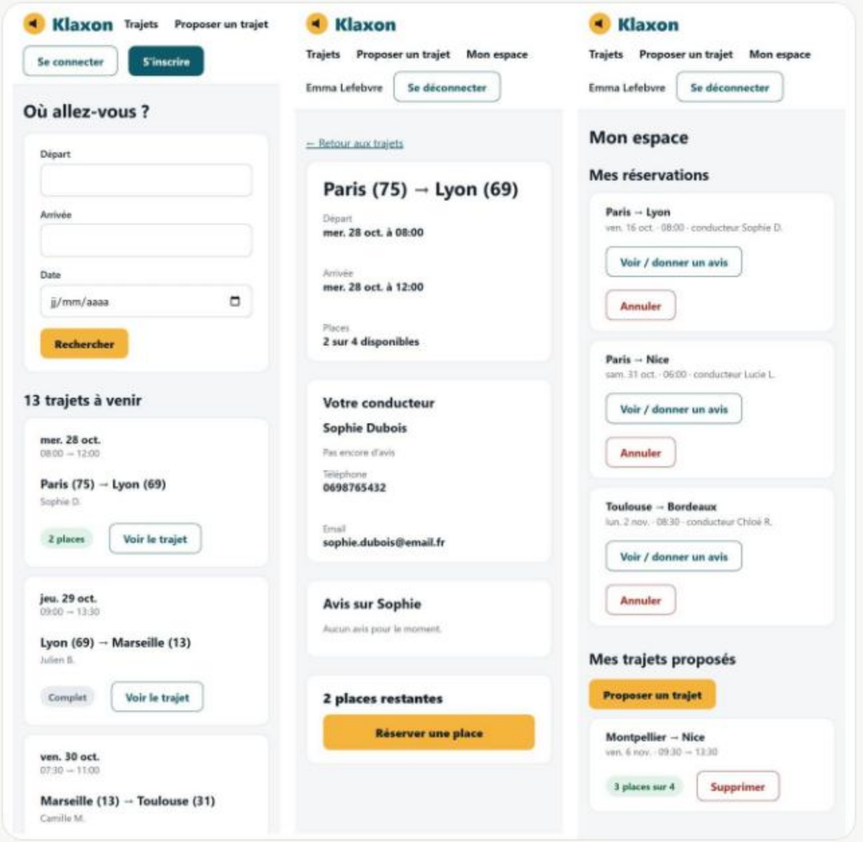
CSS adaptatif

Couleurs en variables, règle spéciale sous 600 pixels

Un composant écrit une fois, réutilisé partout : par exemple, la ligne d’un trajet affiche « Complet » ou le nombre de places.

Le résultat : ordinateur et téléphone





Ordinateur : deux colonnes. Téléphone : les blocs s'empilent.

FRONT-END · COMPÉTENCE 3

Une interface qui réagit

useState

Garder en mémoire : les trajets, un champ, une erreur

useEffect

Aller chercher les données à l'ouverture d'une page

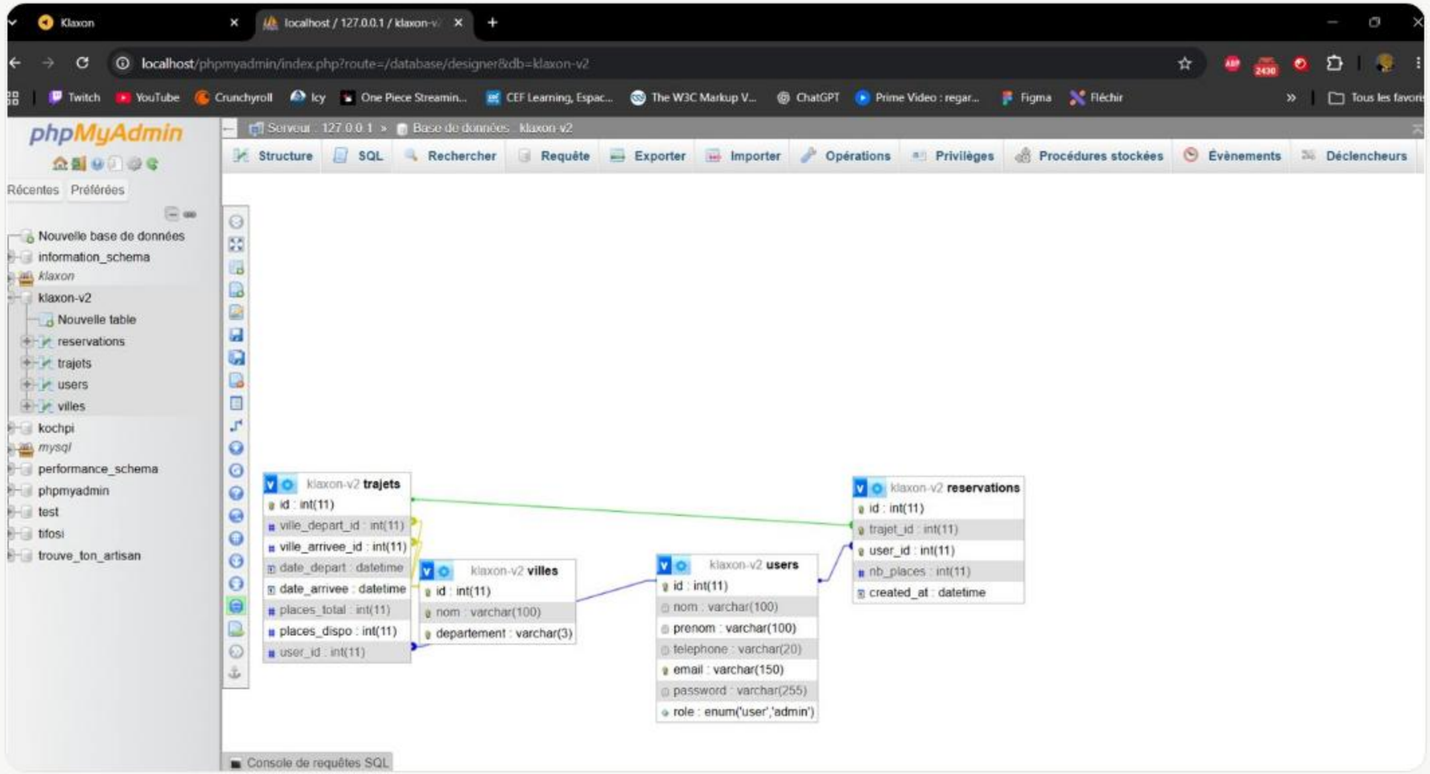
Un contexte

Partager qui est connecté entre toutes les pages

Tous les appels au serveur passent par **un seul fichier**, **api.js**, qui transforme les erreurs en messages lisibles.

BACK-END · COMPÉTENCE 4

Une base MySQL à quatre tables



users et **villes**

trajets : relie deux villes et un conducteur

reservations : relie un passager et un trajet

BACK-END · COMPÉTENCE 4

La base refuse les données incohérentes

Un email ne sert **qu'une fois**

Une réservation par passager et par trajet

L'arrivée est **après** le départ

Entre **1 et 9** places, jamais de places négatives

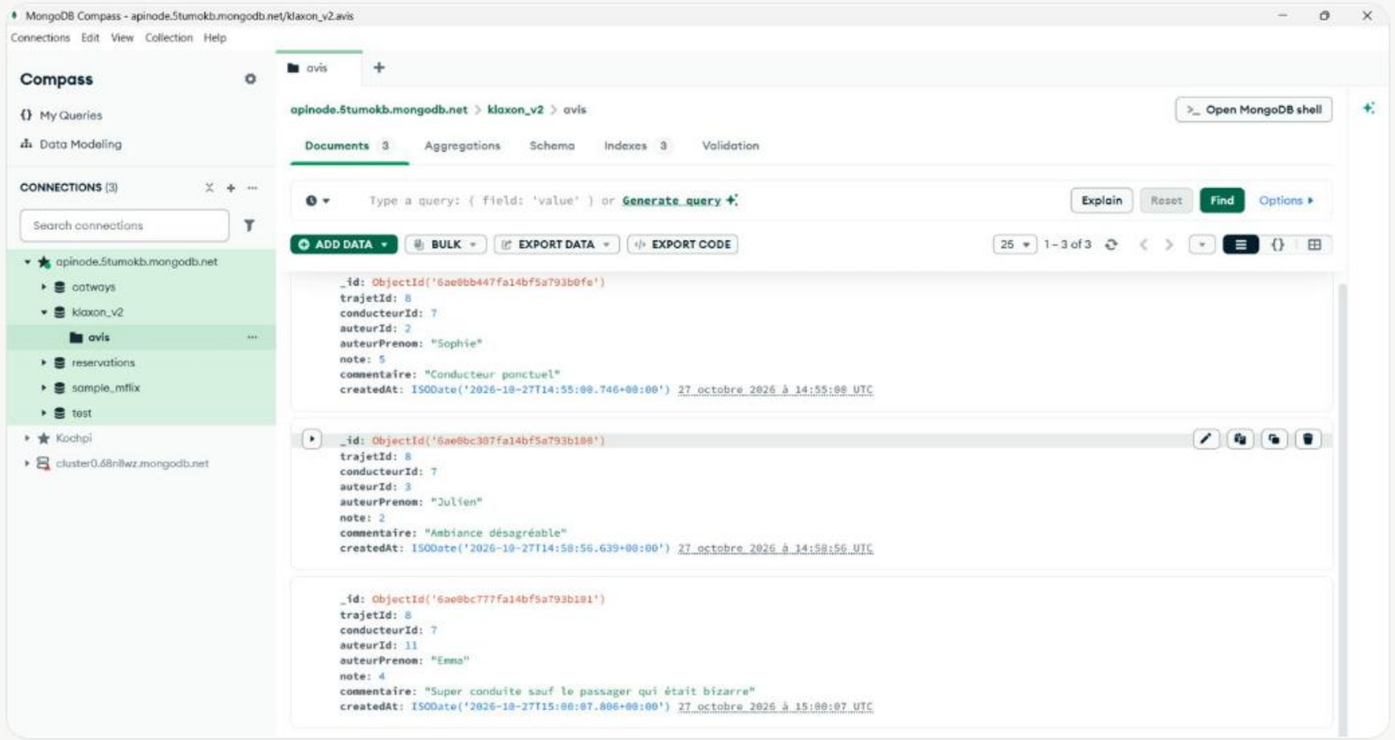
Un trajet supprimé **emporte ses réservations**

Une deuxième protection, **derrière le code** du serveur

BACK-END · COMPÉTENCE 5

Les avis dans MongoDB

```
{
trajetId: 8,
conducteurId: 7,
auteurId: 2,
auteurPrenom: "Sophie",
note: 5,
commentaire: "…",
createdAt: Date
}
```



Un **index unique** garantit un seul avis par passager et par trajet.

BACK-END · COMPÉTENCE 5

Deux façons d’accéder aux données

MySQL

Requêtes **préparées** : les valeurs sont envoyées à part.

```
SELECT … FROM users
WHERE email = ?
```

Aucune injection SQL possible.

MongoDB

La **note moyenne** est calculée par la base.

```
$match : ce conducteur
$group : $avg des notes
```

Arrondie à une décimale.

BACK-END · COMPÉTENCE 6

Réserver : cinq vérifications

1. Le trajet existe
2. Ce n’est pas son propre trajet
3. Il n’est pas déjà parti
4. Il reste une place
5. Le passager n’a pas déjà réservé

Tout est bon : réservation enregistrée, **une place en moins**.

SÉCURITÉ

Une protection à chaque couche

Mots de passe

Hachés avec bcrypt, jamais en clair

Session

Jeton dans un cookie httpOnly et SameSite

Injections

Requêtes préparées, types vérifiés

Force brute

5 échecs maximum par 15 minutes

XSS

Texte échappé par React, en-têtes helmet

Droits

Seul le propriétaire modifie ses données

JEU D'ESSAI · AVIS ET NOTE MOYENNE

12 essais, aucun écart

Notes données : 5, 2 et 4

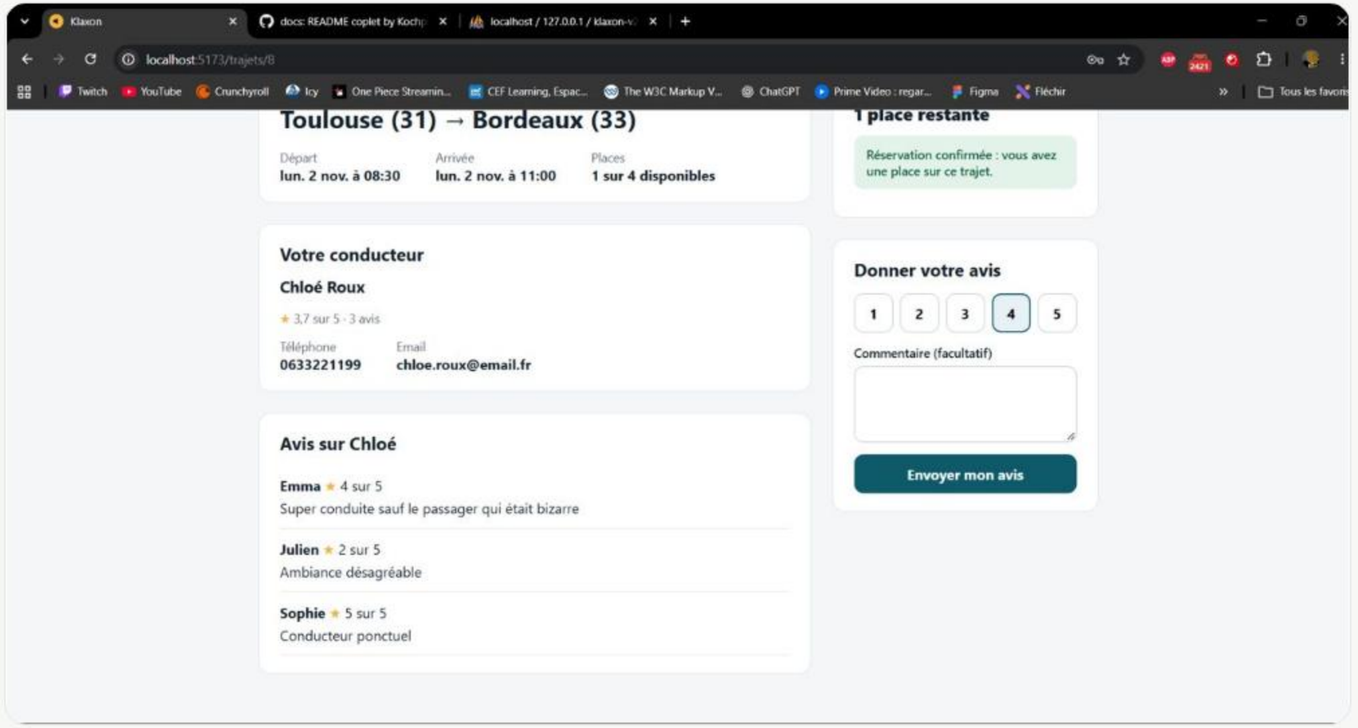
3,7 sur 5

(5 + 2 + 4) / 3 = 3,67, arrondi

Non-passager : refusé (403)

Note 8 ou injection : refusé (400)

Deuxième avis : refusé (409)

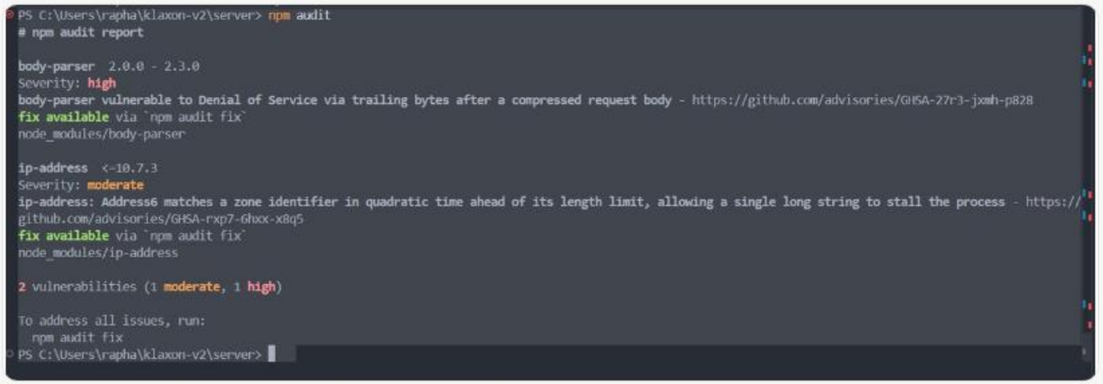


VEILLE DE SÉCURITÉ

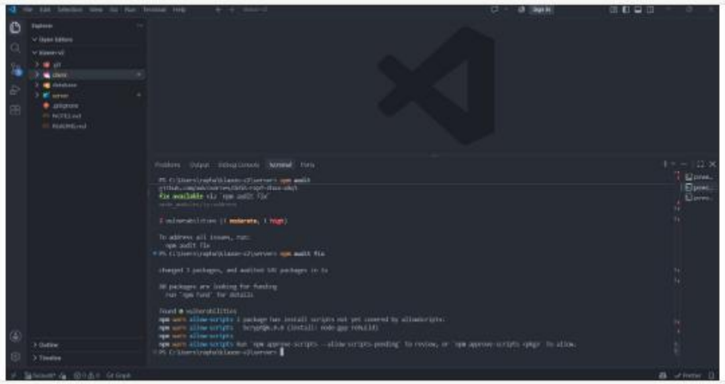
Attaquer pour vérifier : une faille réelle

10 sur 11
attaques simulées bloquées

1 faille
dans les dépendances, corrigée



Avant : 2 failles (npm audit)



Après npm audit fix : 0 faille

DÉMONSTRATION

Klaxon v2 en direct

1

Connexion

Se connecter avec un compte de démo

2

Recherche

Chercher un trajet entre deux villes

3

Réservation

Réserver : une place en moins

4

Avis

Laisser un avis : la note moyenne se met à jour

LIMITES ET ÉVOLUTIONS

Ce que je ferais ensuite

Dernière place

Pas de transaction SQL : deux réservations simultanées pourraient passer

Rôles

Un seul rôle « user » : ajouter un administrateur

Fonctions

Pas de modification d'un trajet, pas de filtre sur les places

Hébergement

Application en local : la mettre en ligne

BILAN

Ce que ce projet m’a apporté

Une application complète : interface, serveur et deux bases de données

Une sécurité pensée dès le début, puis testée par des attaques

Un code sur GitHub, un dossier de projet et des essais documentés

KLAXON V2

Merci de votre attention

Avez-vous des questions ?

github.com/Kochpi/Klaxon-V2